

Descrizione progetto

Vito Mellone
n° matricola: 695232
Università di Pisa
a.a. 2025-2026

1 Inizializzazione e cammini minimax

Per prima cosa voglio risolvere il problema di trovare un cammino minimax ottimo tra due nodi u e v . Per farlo ho idea di procedere così:

1. Creo una classe "Grafo pesato" in cui inserire gli archi tra i nodi e i relativi pesi.
2. Individuo la componente connessa più grande.
3. Per trovare un cammino tra u e v utilizzo un algoritmo ricorsivo che sfrutta una priority queue: il passo base considera tutti i nodi adiacenti a u e li inserisce in coda dando la priorità ai cammini collegati tramite archi con costo inferiore, successivamente considera il nodo con priorità più alta: se è v interrompe il processo e restituisce il costo del cammino, altrimenti reitera considerando anche i nodi adiacenti al quello attuale.
4. Dopo aver trovato un cammino tra u e v , grazie al meccanismo della priority queue so che quello è un cammino minimax ottimo: sia $P_0 = u, x_1, \dots, x_n, v$ il cammino trovato e supponiamo che il suo costo sia M_0 , se per assurdo esistesse un altro cammino $P_1 = u, y_1, \dots, y_m, v$ di costo M_1 tale che $M_1 < M_0$. In P_0 esiste un arco $x_k \rightarrow x_{k+1}$ di costo M_0 e inoltre so che $u \rightarrow y_1$ ha costo $c_{u \rightarrow y_1} \leq M_1 < M_0 = c_{x_k \rightarrow x_{k+1}}$, ma allora nell'esecuzione dell'algoritmo di ricerca l'arco $u \rightarrow y_1$ avrebbe avuto la precedenza su $x_k \rightarrow x_{k+1}$ e quindi il cammino P_1 sarebbe stato trovato prima di P_0 .

1.1 Classe grafo pesato e funzioni ausiliarie

La classe GrafoPesato che ho usato è la stessa che abbiamo dichiarato a lezione. Le funzioni principali che ho utilizzato per questa prima query sono:

- **new_edge** che serve per l'inserimento di nuove relazioni AS, inoltre aggiorna la quantità di nodi effettivi del grafo e di archi.

- **add_weight** serve per definire i pesi degli archi in funzione dei cammini BGP,
- **cc_massima** individua la componente connessa massima del grafico rindicizzando gli archi tramite una funzione similhash in modo da ottimizzare l'uso della memoria
- **costi_minimax** trova i costi di cammini minimi tra un dato nodo di partenza e restituisce il vettore contenente i costi. Il suo algoritmo si basa sullo scheletro dell'algoritmo di dijkstra svolto a lezione il cui funzionamento é garantito dalla dimostrazione al punto 4.

Per le prime due funzioni il costo di esecuzione é $O(1)$.

Il costo di **cc_massima** è: $O(M)$ dove $M := \text{"id maggiore tra i nodi"}$ per la creazione di `visited`. il ciclo `for` visita una volta ogni nodo grazie al vettore `visited`, e anche i confronti tra le dimensioni delle componenti non aumentano la complessità. Il costo della creazione e della rindicizzazione tramite il vettore `id` é sempre $O(M)$, infine l'inserimento degli archi é $O(E)$, quindi il costo totale è $O(M + E)$.

Per **costi_minimax** la costruzione della priority queue e del vettore **visited** costa $O(V)$. Il ciclo `while` esegue un `.pop` alla coda che costa $\log(\text{dimensione della coda})$ e viene eseguito finché la coda non é vuota. Il numero massimo di elementi all'interno della coda, che é uguale al numero massimo di push che vengono eseguiti, è uguale al numero degli archi E : l'utilizzo delle due guardie `visited[x]` e `visited[y]` assicura che il push venga eseguito al più una sola volta per arco, perciò ho al massimo E elementi in coda. Il costo totale é quindi $O(V + E \log E)$.